



Backend Career Roadmap — 2-Month Accelerated Edition

For: Someone who completed Phase 1 → Phase 2.5 (Node.js · TypeScript · Express · PostgreSQL · Prisma · JWT · Layered Architecture)

Goal: Distill Phase 3 → Phase 5 into 2 focused months and walk out job-ready. **Format:** Month 1 = Study · Month 2 = Build



Why This Works

You already know the *patterns* — REST APIs, layered architecture, auth, ORM, and validation. What's left is translating that fluency into the enterprise ecosystem (Java + Spring Boot), adding the infrastructure layer (Redis, async, Docker, CI/CD), and proving you can build something real end-to-end. This roadmap skips the fluff and keeps only what hiring managers actually test for.

1 Month 1 — Accelerated Study Plan



Week 1 — Java + Spring Boot Rapid Ramp

Goal: Get productive in Spring Boot fast. You know the patterns already — this week is about learning the *language and framework syntax*, not the concepts.

Topics to cover:

- Java for backend engineers: types, generics, collections (`List`, `Set`, `Map`), exception handling (`try/catch`), immutability
- Spring Boot project setup via Spring Initializr
- Core annotations: `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PathVariable`, `@RequestBody`
- Dependency Injection + Inversion of Control (Spring's way vs manually wiring in TS)
- Layered architecture: Controller → Service → Repository (you know this, just map it to Java)
- JPA + Hibernate: `@Entity`, `@Id`, `@GeneratedValue`, `@OneToMany`, `@ManyToMany`
- Lazy vs Eager loading — and why N+1 queries will kill you
- DTOs + Mappers: keeping your API contract separate from your DB model
- Connect Spring Boot to PostgreSQL

Weekly Deliverable:

Working CRUD API for a single entity (e.g., a Product resource) with PostgreSQL, using clean layered structure and DTOs.

🔒 Week 2 — Spring Security, Validation & Testing

Goal: Add production-level auth, robust validation, and a basic test suite. These three things come up in almost every technical interview and every code review.

Topics to cover:

- Request validation: `@NotNull`, `@Size`, `@Email`, `@Valid` on request bodies
- Global exception handler with `@ControllerAdvice` — one place to handle all errors, consistent response format
- JWT authentication flow inside Spring Security (you know JWT, now apply it in Spring's opinionated way)
- Authentication filter + Authorization filter — how Spring intercepts requests
- Role-based access control: `ADMIN` / `USER` guard on specific routes
- CORS configuration
- Common attack awareness: SQL Injection, XSS, CSRF — what Spring already protects you from vs. what you need to configure
- Unit testing with **JUnit** + **Mockito**: mock your dependencies, test the service layer in isolation
- Integration testing: test your repository and controller with a real (or embedded) DB

Weekly Deliverable:

Same API from Week 1, now secured with JWT, role-guarded routes, global error handler, and at least 5 passing tests (unit + integration).

⚡ Week 3 — Redis, Caching, Async & Rate Limiting

Goal: Learn what separates a toy backend from a production one. These are the concepts that appear in senior and mid-level interviews even for junior roles. Understanding them — even without deep operational experience — signals maturity.

Topics to cover:

Redis & Caching:

- Redis fundamentals: key-value storage, TTL, eviction policies
- Cache-aside pattern: check cache → on miss, hit DB → populate cache

- What should be cached vs. what should never be cached
- Cache invalidation: time-based TTL vs. event-driven eviction
- `@Cacheable` and `@CacheEvict` in Spring Boot
- Monitoring cache hit/miss ratio (why it matters)
- Cache stampede and how to prevent it

Async Processing:

- Sync vs. async workloads — know which tasks belong in a queue (emails, reports, notifications, file processing)
- Background jobs with `@Async` in Spring
- Retry and failure handling strategies for async jobs
- Basic message queue concepts: producer, consumer, queue vs. pub-sub (conceptual — know the idea, not the full implementation)

Rate Limiting:

- API abuse patterns and why rate limiting is non-negotiable in production
- Implement rate limiting using Redis counters (sliding window, per user, per IP)
- Returning proper `429 Too Many Requests` responses

Performance:

- N+1 query deep dive — diagnosis and fix with `JOIN FETCH` / `EntityGraph`
- Indexing strategy recap
- Pagination: offset vs. cursor — when each one is appropriate

Weekly Deliverable:

API now has Redis caching on at least two high-read endpoints, a simulated async notification job triggered on a status change, and a rate limiter on your public-facing routes.

Week 4 — Docker, CI/CD & Cloud Deployment

Goal: Make everything deployable. This is the single skill that moves you from "junior candidate" to "hire-able backend engineer." Most companies won't let you near production without it.

Topics to cover:

Docker:

- Containerization concepts — why Docker exists and what problem it solves
- Docker basics: images, containers, volumes, networking
- Writing a `Dockerfile` for a Spring Boot application
- Multi-stage builds — keep your final image small
- `docker-compose.yml`: wire up your backend + PostgreSQL + Redis as a stack
- Environment variable management inside containers
- Persisting data with volumes

CI/CD with GitHub Actions:

- What CI/CD actually means and why it matters beyond the buzzword
- Write a GitHub Actions workflow: Lint → Test → Build → Deploy
- Fail the build on test failures — enforcing quality gates
- Build and push Docker images inside CI
- Pull and redeploy on your cloud server

Deployment:

- Deploy to **Railway** or **Render** (both have free tiers, both support Docker)
- Configure environment variables securely in the cloud dashboard
- Health check endpoints (liveness + readiness) — why your container orchestrator needs them
- Graceful shutdown: finishing in-flight requests before the process dies
- Structured logging: logs in JSON format so they're machine-parseable

Weekly Deliverable:

Full Dockerized application running in the cloud. Every push to `main` triggers your GitHub Actions pipeline — tests run, image builds, and new version deploys automatically. No manual steps.

Month 2 — Project: Build "Hireloop"

A Recruitment Pipeline Management Platform for Growing Companies

Business Brief — From the Owner's Desk

"I run a 60-person tech company and we're hiring hard — 8 open positions right now across engineering, product, and design. My HR team is drowning. They're tracking

candidates in a Google Sheet with 14 columns, copy-pasting 'we'll be in touch' emails manually, and chasing hiring managers on Slack for interview feedback that never comes.

Last month we lost a senior engineer — a genuinely great candidate — because nobody remembered to send the offer letter. He'd been in our pipeline for 3 weeks, we forgot to follow up, and he accepted somewhere else.

I need a platform. Something my HR team logs into every morning and knows exactly: which candidates need action today, which positions are moving fast, which are stalled.

Hiring managers should be able to leave feedback without opening their email.

Candidates should know where they stand without emailing us every week.

Here's what I care about:

First — every candidate needs to move through a clear pipeline. They apply, we screen them, we interview, we make a decision. No more candidates sitting in limbo for two weeks because a stage wasn't updated.

Second — hiring managers need boundaries. A backend hiring manager should only see candidates for backend roles. They shouldn't be able to mess with other teams' pipelines. Junior HR coordinators shouldn't be able to delete jobs or override a senior HR manager's decisions.

Third — I want a dashboard. Not a spreadsheet. A real dashboard that tells me: how many open roles, how many candidates in each stage, where the bottlenecks are, and how long on average it takes us to hire someone. If it's taking 45 days to close a backend role and 12 days to close a design role, I want to know that.

Fourth — notifications. Candidates should automatically get an email when they move to the next round or get rejected. Hiring managers should get reminded when they have a candidate waiting for their feedback. I don't want my HR team spending 40% of their time sending status emails.

The system will probably handle 200 to 500 active candidates at once across 20 to 30 open positions. Nothing crazy. But it needs to be fast and it absolutely cannot go down on a Monday morning when everyone's applying after the weekend job posts go live.

One more thing — this is sensitive data. Candidates cannot see each other. Not even accidentally. Build this thing properly."

Feature Breakdown (What the Business Needs)

1. Job Posting Management HR creates, edits, and closes job openings. Each job has a title, department, location, and deadline. Jobs in "published" state are visible to the public. Jobs can be drafted before going live.

2. Candidate Application System External candidates browse open positions and submit their application. Once submitted, the application enters the pipeline at the "Applied" stage. Candidates can log in to check their own application status — nothing more.

3. Recruitment Pipeline Applications move through a fixed set of stages: Applied → Screening → Interview → Final Round → Offer → Hired / Rejected HR moves candidates between stages. Each stage transition can include internal notes and a feedback score. The history of stage changes is preserved.

4. Hiring Manager Portal Hiring managers are assigned to specific job postings. They can only see candidates for their assigned positions. After an interview, they submit structured feedback (rating + written notes). They receive an in-app alert when a candidate reaches their review stage.

5. Automated Notifications When a candidate moves to a new stage, an automated email goes out (simulated in the project — no real SMTP required). When a hiring manager has had a candidate waiting more than 48 hours without feedback, they get an in-app reminder. These run as background jobs — they must not slow down the main API response.

6. HR Analytics Dashboard A single API endpoint that returns: total open positions, candidates per pipeline stage across all active jobs, average days spent in each stage, average time-to-hire per department, and conversion rate from Applied to Hired. HR views this on a read-only dashboard that refreshes every few minutes.

7. Role-Based Access Control Five roles with strictly defined permissions:

- **Super Admin** — full access, can manage users and system settings
- **Senior HR Manager** — full access to all jobs and candidates
- **Junior HR Coordinator** — can view and update assigned pipelines only
- **Hiring Manager** — read candidates and submit feedback for their assigned jobs only
- **Candidate** — can view only their own application status

8. Performance & Reliability The public "apply" endpoint must be protected against spam bursts. The dashboard stats endpoint is read-heavy and should not hit the database on every request. The system must handle at least 300ms response time on core operations. Deployments must not cause visible downtime.

📌 Weekly Project Flow

Week 1 — Architecture & Core Setup

Stand up the project structure, development environment, and CI pipeline before writing a single feature. Design the full data model on paper first. Build authentication for all roles. Implement Job CRUD as the first feature to validate the architecture holds up.

Week 2 — Pipeline & Core Business Logic

Build the candidate application flow and the stage transition engine. This is the core of the product — get it right. Attach hiring managers to jobs. Implement the feedback system. Handle all business rule edge cases: applying to a closed job, moving a candidate backwards, duplicate applications.

Week 3 — Async Features & Performance Layer

Wire in the notification system as a background job. Build the analytics dashboard endpoint. Protect the public apply endpoint with rate limiting. Cache the dashboard stats and the job listing endpoint. Add the candidate's own status view with properly scoped data.

Week 4 — Polish, Deploy & Documentation

Write the full test suite — both unit tests for business logic and integration tests for the API layer. Write Swagger/OpenAPI documentation for every public endpoint. Finalize the Docker Compose setup for production. Deploy to cloud with full CI/CD. Conduct a personal security review: auth, input validation, data scoping. Write a clean README that explains the architecture and how to run it locally.

✅ Exit Criteria — Are You Job-Ready?

Check every box before applying:

- ☐ I can build a secure, layered REST API in Spring Boot from scratch without looking things up
 - ☐ I can implement JWT authentication with role-based access control and explain why each design decision was made
 - ☐ I can integrate Redis for both caching and rate limiting and explain the trade-offs
 - ☐ I can write unit tests with Mockito and integration tests against a real database
 - ☐ I can Dockerize a multi-service application with a `docker-compose.yml`
 - ☐ I have a working GitHub Actions CI/CD pipeline that deploys automatically
 - ☐ I have a live, publicly accessible URL to show in interviews
 - ☐ I can walk through the Hireloop architecture in a whiteboard interview — data model, auth flow, async jobs, caching strategy, deployment
 - ☐ My code on GitHub looks like someone who cares: clean commits, a proper README, no secrets committed
-

Tech Stack You'll Graduate With

Layer	Technology
Language	Java 17+
Framework	Spring Boot 3.x
Database	PostgreSQL + JPA/Hibernate
Cache	Redis
Auth	JWT + Spring Security
Testing	JUnit 5 + Mockito
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions
Cloud	Railway or Render
API Docs	Swagger / OpenAPI
Secondary Stack	TypeScript + Express + Prisma (Phase 2.5 — still valid)

Note: You already have a TypeScript/Node.js backend portfolio from Phase 2.5. By the end of this plan, you'll have a second full-stack backend project in Java/Spring Boot — two language ecosystems, one developer. That is a genuinely strong junior profile.